

PRACTICAL-5

AIM: Implementation of a knapsack problem using dynamic programming.

ALGORITHM:

Knapsack
Algorithm:
Input: Number of items n , knapsack capacity w ,
weights $w[i]$, and values $v[i]$.
Output: Maximum value and selected items.
Step 1: Start.
Step 2: Read n , w , $w[i]$ and $v[i]$.
Step 3: Initialize $dp[i][0] = 0$ and $dp[0][j]$.
Step 4: For $i = 1$ to n and $j = 1$ to w , repeat.
Step 5: If $w[i] > j$,
 $dp[i][j] = dp[i-1][j]$
Step 6: otherwise, $dp[i][j] = \max(v[i] + dp[i-1][j - w[i]], dp[i-1][j])$.
Step 7: Display $dp[n][w]$ as maximum value.
Step 8: Trace the table to find the selected items.
Step 9: Stop.

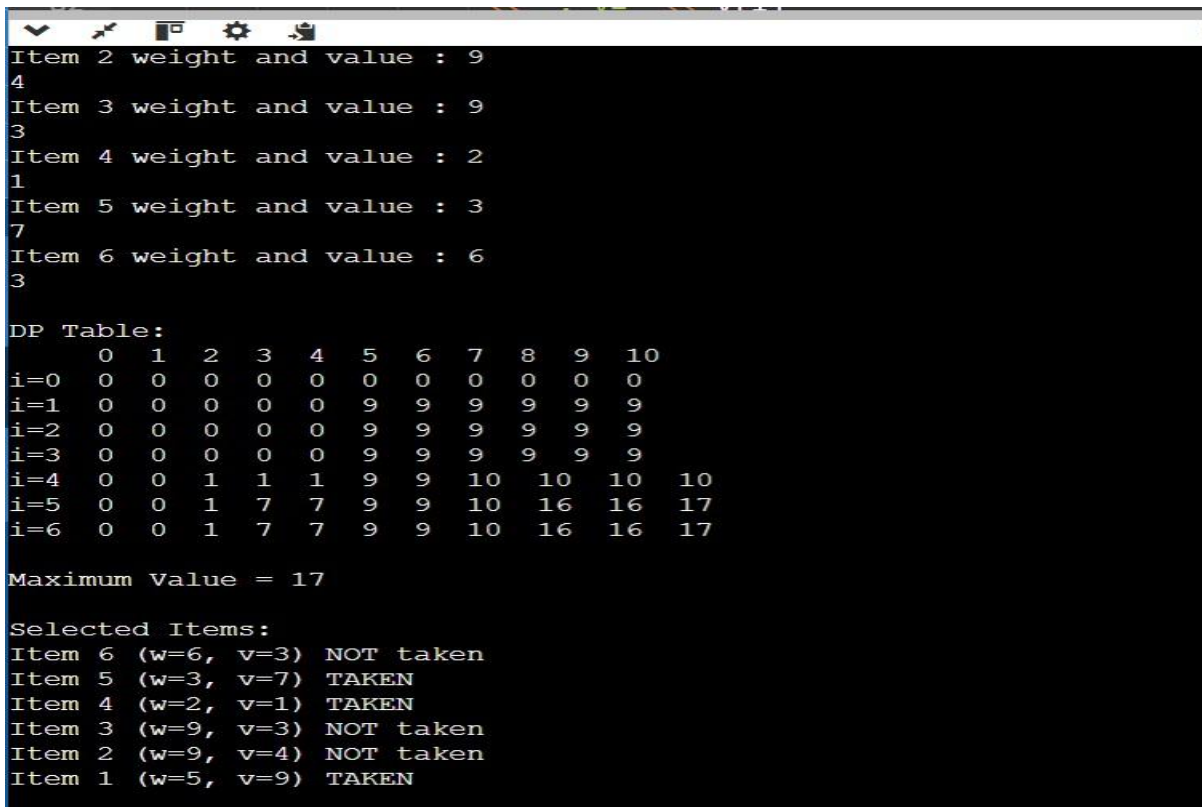
CODE:

```
#include<iostream>
using namespace std;
class Knapsack {
private:
int w[10]; // weights
int v[10]; // values
int n; // number of items
int W; // capacity
int dp[10][100]; // dp table
public:
// — Get Input —
void getInput() {
cout << "Enter number of items : ";
cin >> n;
cout << "Enter knapsack capacity : ";
cin >> W;
for(int i = 1; i <= n; i++) {
cout << "Item " << i << " weight and value : ";
cin >> w[i] >> v[i];
}
}
// — Build dp table —
void solve() {
// base case — row 0 and col 0
for(int i = 0; i <= n; i++) dp[i][0] = 0;
for(int j = 0; j <= W; j++) dp[0][j] = 0;
// fill table
for(int i = 1; i <= n; i++) {
for(int j = 1; j <= W; j++) {
if(w[i] <= j) {
// take or not take
dp[i][j] = max(
v[i] + dp[i-1][j-w[i]],
dp[i-1][j]
);
} else {
// cannot take item i
```

```
dp[i][j] = dp[i-1][j];
}
}
}
}
// — Print dp table —
void printTable() {
cout << "\nDP Table:" << endl;
cout << " ";
for(int j = 0; j <= W; j++)
cout << j << " ";
cout << endl;
for(int i = 0; i <= n; i++) {
cout << "i=" << i << " ";
for(int j = 0; j <= W; j++)
cout << dp[i][j] << " ";
cout << endl;
}
}
// — Find selected items —
void findItems() {
cout << "\nSelected Items:" << endl;
int j = W;
for(int i = n; i >= 1; i--) {
if(dp[i][j] != dp[i-1][j]) {
cout << "Item " << i
<< " (w=" << w[i]
<< ", v=" << v[i]
<< ") TAKEN " << endl;
j = j - w[i];
} else {
cout << "Item " << i
<< " (w=" << w[i]
<< ", v=" << v[i]
<< ") NOT taken " << endl;
}
}
}
// — Show result —
void showResult() {
cout << "\nMaximum Value = "
```

```
<< dp[n][W] << endl;
}
};
// — MAIN —
int main() {
    Knapsack k;
    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();
    return 0;
}
```

OUTPUT:



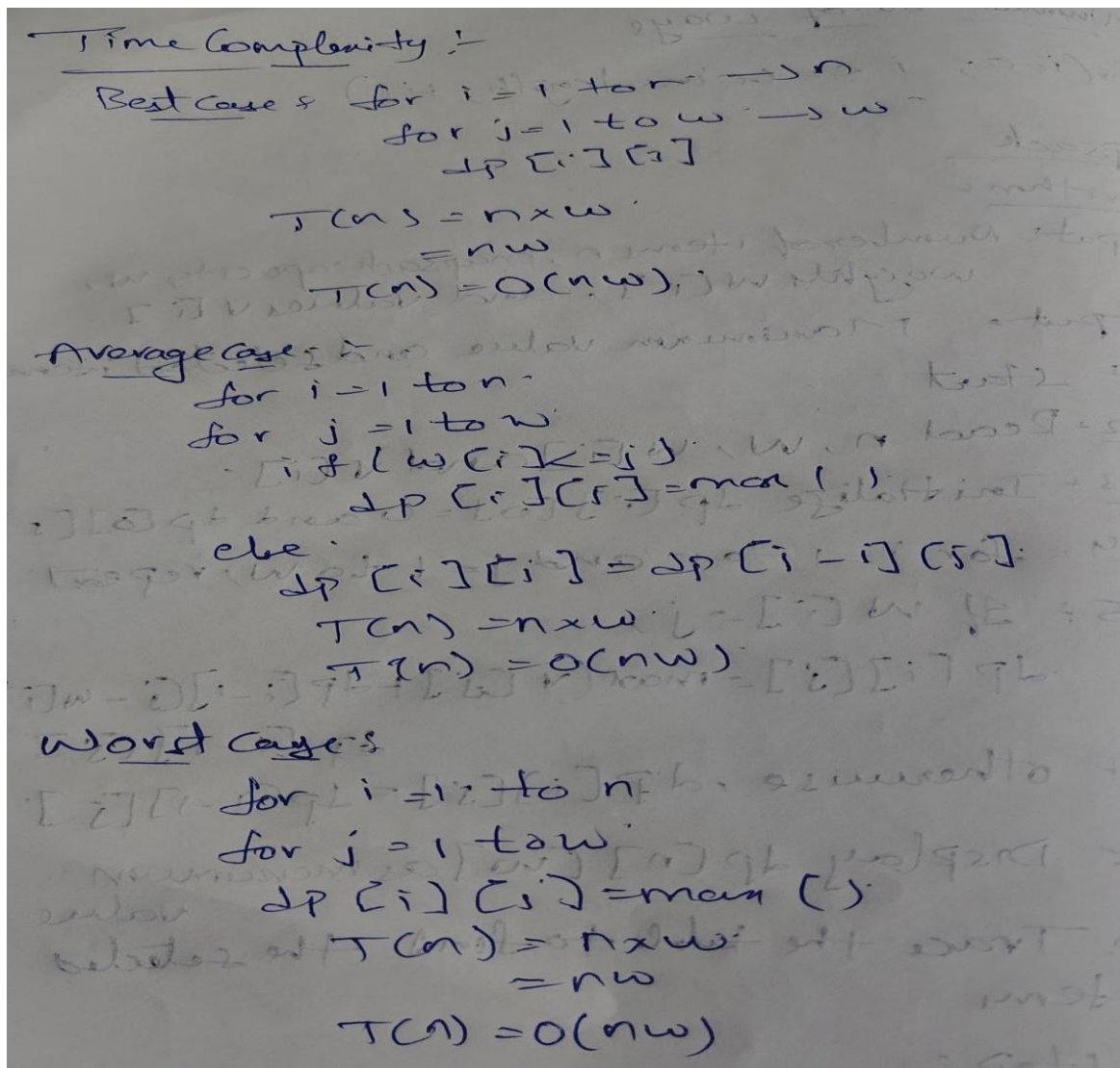
```
Item 2 weight and value : 9
4
Item 3 weight and value : 9
3
Item 4 weight and value : 2
1
Item 5 weight and value : 3
7
Item 6 weight and value : 6
3

DP Table:
   0  1  2  3  4  5  6  7  8  9 10
i=0  0  0  0  0  0  0  0  0  0  0  0
i=1  0  0  0  0  0  9  9  9  9  9  9
i=2  0  0  0  0  0  9  9  9  9  9  9
i=3  0  0  0  0  0  9  9  9  9  9  9
i=4  0  0  1  1  1  9  9 10 10 10 10
i=5  0  0  1  7  7  9  9 10 16 16 17
i=6  0  0  1  7  7  9  9 10 16 16 17

Maximum Value = 17

Selected Items:
Item 6 (w=6, v=3) NOT taken
Item 5 (w=3, v=7) TAKEN
Item 4 (w=2, v=1) TAKEN
Item 3 (w=9, v=3) NOT taken
Item 2 (w=9, v=4) NOT taken
Item 1 (w=5, v=9) TAKEN
```

TIME COMPLEXITY:



CONCLUSION: The 0/1 Knapsack problem was successfully implemented using Dynamic Programming. It finds the maximum value within the given capacity efficiently by avoiding repeated calculations.